

INTRODUCTION

Dans le cadre d'un TP relatif au cours d'Optimisation, plus précisément au duo formé par celui de 'Recherche Locale' et de 'Programmation Génétique', mon binôme et moi reçûmes la délicate mission de résoudre un problème à la fois informatique, algorithmique, d'optimisation (recherche locale – algorithmique génétique) et artistique. En effet, notre tâche était de mettre au point un programme python qui ; avec un certain nombre de paramètres pris en entrée (ligne de commandes type console), dont notamment une image source, une indication de forme géométrique, un nombre maximal de ces formes, un temps maximal d'exécution ... ; devrait générer un SVG qui ne serait autre qu'un amas desdites formes, mais intelligemment et astucieusement, optimalement, disposées, de sorte à ressembler le plus possible à l'image de départ. Cette tâche complexe serait l'occasion pour nous de faire parler à la fois notre créativité, notre ingéniosité, nos capacités à résoudre des problèmes d'optimisation de la vie réelle, autant que celle de pouvoir résoudre une tâche de recherche locale et de programmation génétique. Ce serait, de plus, l'opportunité pour nous, d'explorer notre façon d'approcher un problème, de l'appréhender, de le comprendre, de tester plusieurs solutions relatives à lui, d'avoir le recul nécessaire pour prendre les bonnes décisions, jusqu'à atteindre le graal : le moment où nous élisons la ou les solutions finales, qui devront évidemment être les plus optimales et satisfaisantes possibles.

Le cadre à présent dressé, les lignes suivantes relateront notre grande aventure dans le monde de la recherche locale, et de celui de l'algorithmique génétique, mettant en lumière nos idées, inspirations, intuitions, chemins pris et les pensées derrières ces décisions, nos doutes, appréhensions, visions du problème, de ses solutions, nos choix, techniques utilisées, etc., et surtout nos propositions finales de solutions à ce problème d'art, à savoir la grande vedette du projet : notre livrable.

OBJECTIFS

Trois objectifs principaux furent attendus de nous :

- 1 – La formulation et la résolution de problème complexe d'optimisation de la vraie vie.
- 2 – L'implémentation d'une approche métaheuristique effective et efficiente
- 3 – L'exploration de l'impact du 'tuning' de paramètres, et probablement d'hyperparamètres, relativement à une ou à des métaheures données.

Pour mieux approcher notre problème à résoudre, il fut congru de, dans un premier temps, prendre déjà le temps de nous interroger sur ledit problème, de le comprendre, autant les défis qu'il soulève, les outils à notre dispositions pour le traiter, le matériel à traiter, les contraintes, les paramètres et les axes possibles impliqués.

Modélisation du problème

La modélisation de notre problème impliqua de répondre, dans un premier temps, à certaines questions, ou en tout cas d'être sûr que nous disposions des bonnes réponses à des questions essentielles auxquelles nous avions possiblement déjà implicitement et inconsciemment, automatiquement fixé réponses. Il fallait être sûrs que nos premiers pas se faisaient dans la bonne détection. Ainsi, nous nous demandâmes :

1 – A quelles données avons-nous accès pour résoudre ce problème ?

Etant donné que le problème que nous devons résoudre implique un traitement d'image, il nous vint naturellement comme réponse que les données à notre disposition se trouveraient être toute image existante et exploitable. Du PNG et du JPEG notamment (le format le plus populaire). Ce fut alors l'occasion de déjà télécharger certaines images (libres de droit) afin de disposer d'une bonne variété de données pour tous nos tests et ajustements. Nous priorisâmes les images artistiques, en particulier la Joconde – qui semblait d'ailleurs, justement, particulièrement appréciée par le professeur comme support d'exemple et d'illustration. En résumé, notre donnée principale est toute image source, en d'autres termes, un gros sac de pixels. C'est ces données là que nous devrions exploiter dans un premier temps.

D'un autre côté, il existait certes les données de base, mais d'autres données entreraient en ligne de compte, elles aussi, durant le traitement de cette donnée source. Il s'agit de supports SVG et PNG (le résultat final serait sous la forme de SVG mais aussi de PNG. Ces supports permettraient, également, des visualisations fréquentes durant le processus, afin qu'il soit permis, à nous développeurs, mais aussi à tout utilisateur final, de suivre l'évolution de tous les corps impliqués dans le dressage du résultat ultime). Bien évidemment, ces supports SVG et PNG ne seraient que les toiles de fond, la scène de théâtre qui n'aurait que pour rôle de mettre en lumière les réelles stars du programme : Les formes à manipuler.

Comment représenter des formes dans un programme informatique ? Et bien c'est là qu'entre en lice la notion de génotype. En effet, là où les formes visuelles représentent le phénotype, le génotype est la façon, en termes de données, dont nous devrions représenter ces formes, les mémoriser, afin de pouvoir les manipuler. Toute manipulation du génotype influe sur le phénotype.

Dans notre cas, il fallait décider de la façon dont notre génotype serait constitué, de ce dont il aurait l'air. Et justement, comme les lignes ultérieures le montreront, cette représentation évolua plusieurs fois durant les phases du projet, selon les axes choisis, les directions prises, et etc.

Toujours est qu'il demeura deux données principales : le sac à pixels (image source constituée de tableau de nombres entre 0 et 255 par 3 canaux couleurs R, G et B) et le génotype (structure de données à manipuler pour atteindre le phénotype-solution).

2 – Quelles sont nos variables de décision

Dans notre cas, les variables de décision furent naturellement associées aux formes que nous devrions manipuler. Ainsi, il nous fût naturel de considérer que ce sont : **la taille de la forme, la position de la forme, la rotation de la forme, ses dimensions, sa couleur, sa transparence**, potentiellement son **z-index**. En effet, c'est en jouant sur ces paramètres que nous modifierions le phénotype, et nous rapprocherions ou éloignerions ainsi donc du résultat optimal et/ou final.

Nos variables de décisions confirmées, il fallait clairement identifier un autre méta-paramètre très fortement corrélé à celles-ci : les contraintes.

3 – Les contraintes de notre problème

Ici, les contraintes principales que nous identifîâmes nous étaient directement fournies par les paramètres d'entrées du programme, donc par l'utilisateur.

A savoir le type de forme à utiliser (il fallait d'ailleurs prévoir un nombre limité de forme, dont l'une d'entre elles qui serait sélectionnée par défaut si jamais la chaîne de caractère associée au paramètre de forme n'est pas reconnue), le nombre de formes à utiliser (là aussi, il fallait penser à limiter ce nombre

que ce soit par borne positive ou borne négative, afin d'éviter par exemple un nombre trop faible ou négatif, ou un nombre bien trop élevé), la durée maximale de calculs (aussi à borner).

Comme toute bonne optimisation sous contrainte, notre solution devrait, à chaque fois, satisfaire chaque contrainte, tout étant le plus optimale possible, c'est-à-dire satisfaire au mieux la fonction objectif. Fonction qui devait justement être clairement et soigneusement choisie : c'est elle, après tout, qui serait la boussole nous guidant vers le graal. Une boussole cassée ne ferait rien d'autre que nous conduire à notre perte avant même le premier pas. Cette étape était donc cruciale.

4 – Fonction objectif

Dans un problème de recherche locale/d'algorithmique génétique, **fonction objectif** rime généralement avec **fitness**. Dans notre cas, la première ne consiste à rien d'autre qu'à minimiser la fitness entre l'image source et l'image finale. En d'autres termes, sachant que quand on parle d'image on parle de pixels, il s'agit de s'assurer que la différence entre les pixels et l'image de départ et de celle finale est le plus petit possible.

La notion de **fitness** et de **loss** sont deux faces de la même pièce. Et durant notre travail, nos axes, directions et chemins empruntés, nous prîmes plaisir à explorer différentes stratégies relatives à ces concepts.

Il a après tout s'agit de parvenir à identifier les meilleures stratégies allant le mieux possible avec les choix techniques courants effectués, au moment de ces choix courants, et selon les méthodes élues.

Nos objectifs et notre modélisation du problème maintenant présentés, débutons notre récit par l'exploration de nos premiers pas, premiers tâtonnements, vers nos premières idées, vers nos premières solutions.

I- PREMIERS PAS

La première chose que nous commençâmes par faire fut de nous lancer dans une phase de compréhension de la structure d'un SVG, et de manipulation de SVG via python. En effet, l'une de nos premières idées consista à envisager d'utiliser comme génotype un fichier SVG contenant nos formes, formes que nous manipulerions directement à travers le fichier SVG en question. Ledit fichier serait alors à la fois notre génotype, mais aussi notre phénotype (lorsqu'il est exécuté).

A ce moment-là, une première stratégie de recherche locale (car la recherche locale fut le premier chemin que nous empruntâmes) fut pensée pour suivre la recette suivante :

- Saisir le fichier SVG
- Le parcourir
 - Pour chaque forme présente dans le SVG :
 - Effectuer une légère mutation aléatoire (rotation ou dimension ou position ou couleur ou transparence)
 - Si fitness améliorée, on passe à la forme suivante, sinon on recommence toujours de façon aléatoire (en se donnant évidemment un nombre limité d'essais sur la même forme avant de quand même passer à la suite. Cela, afin d'éviter une boucle infinie dans un cas où aucune mutation sur la forme courante n'améliorerait la fitness)
- Répéter l'opération plusieurs fois.

Lors de nos recherches pour comprendre la structure du SVG et ses principes de manipulations en python (qui furent d'ailleurs le lieu et l'occasion de premiers tests, comme par exemple la génération de SVG vides à fond noirs, le dessin d'ellipses, de rectangles, de croix, de formes irrégulières, etc.), nous nous rendîmes compte que pour effectuer une rotation sur une forme SVG, il était crucial de spécifier les coordonnées du centre de rotation. Une idée de stocker, alors, en parallèle, les coordonnées de chaque forme, dans une structure de données externe émergea (elle visait nul autre que la performance, puisque sinon il nous aurait fallu d'abord devoir lire les paramètres de propriété de la balise concernée, récupérer les coordonnées de position qui nous intéressent avant de pouvoir les utiliser). De la sorte, pour une forme donnée, prenons l'exemple d'une ellipse par exemple, nous pourrions directement en obtenir les coordonnées du centre (que nous mettrions d'ailleurs à jour quand la forme changerait de position) afin de pouvoir tranquillement réaliser ladite rotation.

Cependant, de premières recherches et de premiers tests nous décourageai rapidement, dans la mesure où une telle manipulation et gymnastique était inutilement coûteuse. D'autant plus que l'idée d'utiliser le SVG lui-même comme génotype nous était venue dans une politique d'optimisation de ressources et de minimisation de quantité de mémoire et structures utilisées. C'était pour nous la solution du 'une pierre deux coups'.

Il naissait alors une grosse contradiction, puisque nous nous prenions à envisager une structure supplémentaire en complément, dans un cadre d'opération de manipulation coûteuse de SVG (lecture du fichier, parcours de son arborescence, identification des bons attributs, réécriture des zones à modifier, ainsi de suite), cela sachant qu'une telle manipulation devrait se faire des millions de fois à la suite dans des centaines voire milliers d'itérations. De plus, ajoutons que le projet de base consistait, en plus de manipuler les formes directement via le xml du SVG, à convertir, à chaque fois, le SVG en PNG (donc en sac à pixels) afin de pouvoir en calculer la fitness comme une somme des différences de pixels avec l'image cible. Même si nous avions pour idée de ne calculer/recalculer que dans la zone où s'était produit le mouvement, il était rapidement évident que là n'était pas le problème. Et nous ne parlerons même pas du calvaire que cela aurait été de réaliser des clonages de tel génotype.

Bref, nous comprîmes vite que le principe d'utiliser du XML d'SVG directement comme sac à génome était une mauvaise idée et l'abandonnâmes.

Il était alors désormais heure et lieu de rapidement et urgemment devoir trouver une autre solution de création de nos formes, mais aussi de déjà décider de la façon propre et performante dont nous représenterions notre génotype.

Cela dit, quelle que soit la stratégie génotypique que nous adopterions, et puisque le SVG demeurerait malgré tout le support (l'un des deux mais le principal) de visualisation de nos phénotypes dans tous les cas, il fallait de toute manière prévoir que notre tactique de dessin de formes nous permette de facilement et efficacement créer du SVG quand besoin à partir de notre génotype.

C'est ainsi alors que guidés par nos recherches, nous nous retrouvâmes à hésiter entre la bibliothèque PILLOW (proposée par le professeur) et OpenCv (d'ailleurs utilisée par ce dernier), prise par ces recherches. Ces deux bibliothèques permettent de créer des formes en python, plus précisément en numpy (de façon 'native' pour OpenCv, et facilement transcritible pour PIL), ce qui est très bon pour les performances (calcul de fitness notamment, qui pourrait par exemple simplement, directement et immédiatement se faire sur base de simples différences entre cellules de matrices : du pain béni pour ce qui est du coût de calcul). Sans parler de combien elles permettent facilement d'exploiter du PNG/JPG ou autre format d'image similaire, que ce soit en lecture comme en écriture, ou de 'jouer' avec les pixels.

Notre choix se porta finalement sur openCV, dans ce premier temps-là, justement et encore une fois pour des raisons de performance, puisque d'après nos premières recherches, openCV battait largement

PIL sur ce terrain-là. Et ayant une idée de la quantité de calculs nécessaires dans des cadres comme le nôtre, de recherche locale, la performance est un luxe que nous ne pouvions nous permettre de minimiser.

Les premiers tests de dessin avec openCV furent d'ailleurs l'occasion de construction des premières esquisses de notre fichier '**shapes.py**', fichier qui contiendrait la classe de formes : une classe mère, dont hériteraient toutes celles des formes prévues. Manipuler nos formes par programmation objet fut, en effet, l'idée qui nous parut la plus naturelle. Ainsi, chaque classe de chaque objet de forme utilise un certain nombre de paramètres en construction qui permettent justement de définir les valeurs des variables de décision le concernant (rotation, position, dimension, couleur, etc.). Puis, par simple appel à une méthode '**draw**' (l'une des méthodes que nous mîmes au point pour les classes de forme), l'objet est dessiné sur un canvas fournit en paramètre.

De plus, pour chacune de ces classes, nous prévîmes une méthode '**get_svg_tag**' qui a pour tâche de fournir, sous forme de chaîne de caractère formatée, le code XML-SVG correspondant à ladite forme, avec les paramètres actualisés dans la balise de l'objet concerné. De la sorte, par simple appel à cette méthode, nous disposions directement de de quoi construire du SVG à partir d'une forme.

Notons, cela-dit, que contrairement à PILLOW, openCV ne permet pas de gérer de façon native la superposition des transparences (si deux formes transparentes et colorées sont dessinées l'une sur l'autre la zone de chevauchement ne prend que la couleur de l'élément au dessus – celui dessiné en dernier –, là où l'on ne pourrait que souhaiter que les deux couleurs de la zone de chevauchement en forme une nouvelle selon la logique de superposition de couleurs). C'est ainsi donc qu'il fallut trouver une astuce. Que nous trouvâmes en scrutant l'état de l'art.

Elle se résume en la méthode suivante :

```
def _apply_transparency(self, canvas, overlay):
    """Méthode interne pour appliquer le mélange Alpha (Blending)"""
    alpha_norm = self.alpha / 255.0
    cv2.addWeighted(overlay, alpha_norm, canvas, 1 - alpha_norm, 0, canvas)
```

Grosso modo, cette dernière convertit la valeur d'alpha (entre 0 et 255) en un coefficient normalisé entre **0.0 et 1.0**. Puis mélange l'image 'overlay' et l'image 'canvas' (qui est donc notre toile principale avec toutes les formes potentielles précédemment dessinées) selon la formule :

Résultat = overlay × alpha + canvas × (1 - alpha).

En résumé, nous disposions de classes fonctionnelles prêtes à nous permettre de dessiner, de contrôler les paramètres de nos formes, de transformer nos toiles à formes en PNG/JPEG (grâce aux méthodes natives de openCV), mais aussi en SVG. Et nos premiers tests furent justement un succès.

Nous pouvions dessiner, superposer les formes et générer les phénotypes. Mais la partie la plus importante restait encore à concevoir : le génotype.

Nos premiers pas étaient couronnés de succès. Nous savions les faire. Il fallait maintenant apprendre à marcher.

II- PREMIERES MARCHES

La première idée de génotype que nous eûmes fut que ce dernier serait une longue liste, dont chaque ligne ne serait autre qu'un tableau de flottants (donc de paramètres) correspondant chacune aux paramètres d'une forme.

Avec le choix d'un tel génotype, nous eûmes idée d'adopter une approche d'algorithmique génétique.

En effet, dans la continuité de l'idée d'algorithme que nous avons évoquée plus haut (celle de parcourir plusieurs fois le génotype et de perturber dans l'ordre chaque forme jusqu'à améliorer la fitness avant seulement de passer à la suite, tout en se donnant un nombre limité d'essais sur la même forme), l'idée impliqua de justement lancer 'en parallèle' plusieurs branches exécutant ledit algorithmique. Python ne permettant pas le parallélisme, il faudrait probablement le faire successivement. Mais en gros, le but serait de partir d'un génotype G, d'exécuter un nombre X fois l'algorithme évoqué à partir de G pour à chaque fois un même nombre d'itérations, puis de garder le résultat avec la meilleure fitness : le génotype H. Il ne resterait alors plus qu'à recommencer le même processus avec H, et ainsi de suite.

Même si le principe génétique impliqué nous paraissait intéressant, cette idée nous semblait avoir un défaut. En tout cas, nous décidâmes de la garder dans un coin de la tête et de peut-être la tester plus tard dans le cas où nous aurions du temps. Mais pour l'heure, nous ne nous contenterions que d'une seule branche (plutôt qu'exécuter X fois et choisir le meilleur, juste exécuter une fois, sur une ligne droite ; notre stratégie de **round-robin** nous paraissant déjà plutôt prometteuse à elle seule).

Ainsi, notre idée de génotype se précisa par le fait que plutôt que de se contenter de stocker uniquement pour chaque ligne les paramètres en flottants, il serait plus efficace de stocker directement les objets de formes en eux-mêmes.

L'exécution d'une perturbation de forme ne consisterait alors simplement qu'en l'appel d'une méthode dédiée sur l'objet de ladite forme.

Parallèlement à ces configurations, nous programmâmes, évidemment, que la taille de la taille de la toile (phénotype ; donc la taille de matrice numpy) serait exactement celle de l'image de départ en pixels. Cela, avec le fait que nous redimensionnons automatiquement (en conservant le ratio) les dimensions de l'image source, si nécessaire, pour s'assurer qu'elles n'excèdent pas une certaine valeur. Pour la santé de notre performance. 1000*1000 fut la valeur que nous choisîmes dans un premier temps (encore naïfs).

D'un autre côté, il a s'agit de mettre au point une fonction de génération aléatoire de formes (point de départ où pour X formes F exigées par la contrainte de départ, X formes F seraient générées avec des paramètres absolument aléatoires sur une toile à fond noir) ; de méthode (de classe de forme) de clonage d'une forme (utile pour le clonage de génotype notamment).

Ce fut donc le lieu et le moment d'exécuter un premier test de génération aléatoire, qui se couronna de succès. La génération aléatoire était au point (nous la testâmes avec pour paramètres 100 rectangles), autant que la transcription de génotype à phénotype (SVG ou PNG)



(Respecte les dimensions de l'image source).

Dans la continuité de l'idée de clonage de forme, nous mîmes au point une fonction de clonage de génome qui aurait pour rôle de parcourir un génome et de cloner chaque forme s'y trouvant, pour finalement retourner une copie parfaite du génome de départ.

Notre système applicatif prenait de plus en plus forme, autant qu'émergeaient, de lignes de code en ligne de codes, les outils de traitement de notre problème qui constitueraient ce système. Seulement, l'un des plus importants de ceux-ci, peut être même le plus important, n'avait encore pas été fait. Il en était le moment...

*Notre fonction de fitness

Sa première version consista à, à partir de deux tableaux numpy multidimensionnels d'images reçus en paramètres, calculer, en parcourant chaque pixel/cellule-nombre de l'image/tableau A, la différence avec le pixel correspondant de l'image B ; puis à cumuler le carré de ces différences.

En soit, il s'agit plus là d'une 'loss' que d'une fitness, mais comme dit au plus haut : généralement les deux faces d'une même pièce.

Précisons, néanmoins, que notre algorithme utilisant cette fitness devrait, selon qu'il considère dans sa logique une fitness meilleure qu'une autre si elle est plus grande ou plus petite, faire intervenir une inversion de signe ou non.

D'un autre côté, nous réalisâmes rapidement que de maintenir comme seuil de dimensions une valeur de 1000*1000, donc de généralement travailler en centaine de milliers de pixels à chaque calcul, était une catastrophe en termes de performance. Il s'agit donc de passer de 1000*1000 à 250*250.

Et toujours dans une optique d'amélioration de performances, nous passâmes à une stratégie de vectorisation de nos matrices de pixels pour le calcul des différences, plutôt que de la calculer à la main pixel à pixel. La vitesse de calcul s'en retrouva drastiquement améliorée.

D'un autre côté, il a s'agit, d'étoffer nos méthodes de classes de formes (nos formes élues étant à ce moment-là les ellipses, rectangles et triangles). Par exemple, par l'ajout d'une méthode '**perturb**' à la classe mère que chaque classe fille serait forcée de devoir implémenter. Cette méthode aurait pour rôle de s'assurer qu'une perturbation aléatoire de la forme concernée ait lieu. Légère rotation, ou léger déplacement, léger redimensionnement, légère variation de couleur sur un canal choisi au hasard, de

transparence, etc. Notons, d'ailleurs, qu'il y a 50% de chances que la perturbation concerne la couleur/transparence, et autant de chances qu'elle concerne plutôt la géométrie.

En supplément à cela, et à des fins de test, nous confectionnâmes une fonction de perturbation total de génotype. Elle parcourt chaque forme du génotype et la perturbe une fois, avant de passer à la suite. Nous essayâmes une itération de perturbation, 2, 10, 100. Le but était de s'assurer que notre code est capable de correctement et de façon cohérente perturber tout génome, donc par la même occasion et par système d'engrenage tout phénotype.

Ce fut un succès.

Nous disposions de tout ce dont nous avons besoin pour mettre en place le premier et réel test total de notre dispositif. Nous savions marcher. Qu'en était-il de savoir courir ?

III- PREMIERES COURSES ET PREMIERES CHUTES

L'algorithme que nous mîmes en place, celui qui aurait pour rôle d'être la colonne vertébrale de notre application, fut, à quelques détails près, celui évoqué plus haut.

- On génère un génotype aléatoire au phénotype ayant les dimensions de l'image de départ (ajustée à 250 pixels de bords maximum – ratio de base conservé)
- On parcourt le génotype
- Pour chaque forme :
 - On fait une perturbation aléatoire sur un clone du génome. Si la perturbation améliore la fitness, on avance après avoir remplacé l'original par son clone, sinon, on recommence avec un autre clone, toujours de façon aléatoire (rotation, dimension, couleur, etc.,). Le but est obtenir quelque chose qui améliore la fitness. Un paramètre de renoncement est mis par défaut à 50 (peut-être modifié comme argument de la fonction). Si au bout de ces 50 essais aucune perturbation de cette forme ne s'est voulue améliorante, on passe à la suivante. (Une idée serait, plutôt que de simplement passer à la suivante, d'effectuer une perturbation brutale, peu importe son impact sur la fitness, avant de passer à la prochaine. Ou alors, de détruire la forme courante, et d'en générer une toute nouvelle aléatoirement. Mais nous n'explorâmes aucune de ces option, par manque de temps, ce que nous regrettâmes).
- Après chaque époque, on met à jour les fichiers SVG et PNG de phénotypes. On peut donc suivre en temps réel l'évolution.
- On recommence autant de fois que spécifié en paramètre.

Notre premier test consista à ne faire qu'une seule époque. Tout sembla fonctionner.

Avant d'augmenter le nombre d'époques, nous ajoutâmes à cette fonction principale de quoi nous permettre de surveiller l'évolution de la fitness (affichage de la fitness à chaque nouvelle époque, qui devrait, d'après la logique codée, justement n'en être que meilleure d'époque en époque), cela avec objectif de veiller à la bonne convergence.

C'est là que survint brutalement notre première vraie chute.

*La chute

Au lieu de converger, on assistait clairement à une divergence. Ou alors, on convergeait, puis on divergeait, puis on reconvergeait, et ainsi de suite. Cela n'avait aucun sens.

Nous gérons pourtant correctement l'inversion de signe de résultat de notre fonction de calcul de fitness, cela en accord avec une logique correcte d'évaluation de fitness et de sélection. Alors où était le souci ? Nous étions perdus. Nous avions beau ausculter notre fonction, toutes celles auxquelles elle faisait appel. Aucun bug ne fut descellé. Et cela était normal, puisque le souci ne se trouvait aucunement là, chercher des minutes ou des heures ne l'y aurait pas fait pousser. Le vrai souci, celui qui causait ce enchainement d'illogisme, se trouvait à la base même du programme : au niveau du traitement de l'image source.

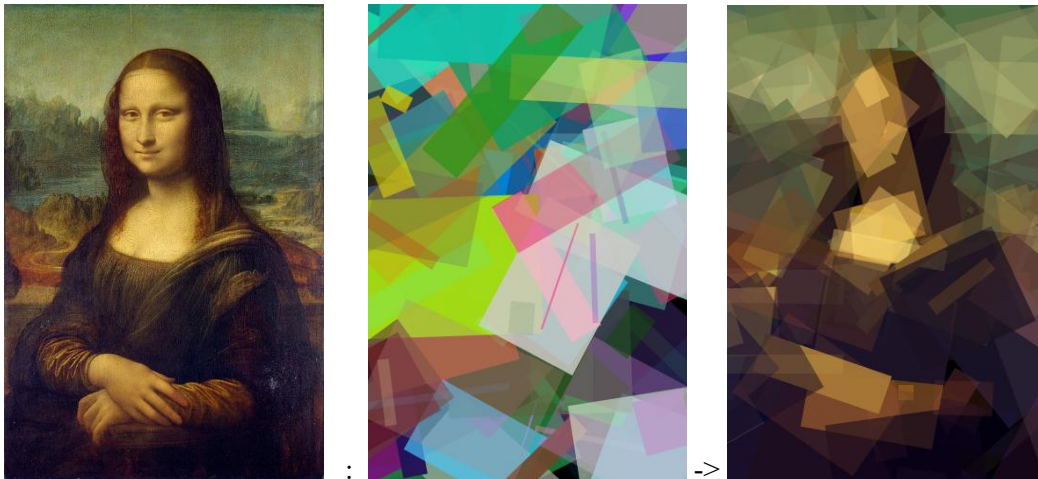
```
target_matrix = np.array(input_image, dtype=np.int16)
```

Ceci est le code de transformation de l'image source en tableau numpy. Comme l'exige apparemment la norme et l'état de l'art il semble, il était nécessaire de coder sur du 16bits. Sauf qu'à travailler avec des valeurs qui peuvent facilement s'envoler (calcul de fitness), il y a très rapidement un débordement, ce qui ne peut être que sujet de comportements inattendus et imprévisibles. Cela, d'autant plus quand on travail avec des signés.

La solution fut donc le castage, au sein de la fonction de calcul de fitness, en 64 bits.

Cette simple action corrigea tout ce chapelet d'illogisme.

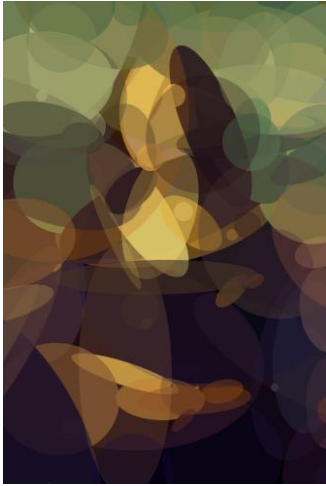
IV- LE RETOUR EN COURSE (PLUS FORTS)



Voici ce que nous obtînmes au bout de 100 époques, avec 100 rectangles, notre moteur de convergence enfin réparé. Notre application fonctionnait. Le résultat était excellent et nous emplis de fierté.

Nous profitâmes de notre joie et euphorie pour ajouter le système de timer imposé par les contraintes du problème.

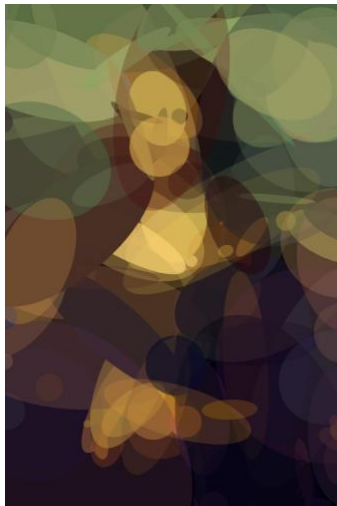
Avec des ellipses, et toujours 100 formes, voici ce que nous obtînmes au bout de 30 minutes :



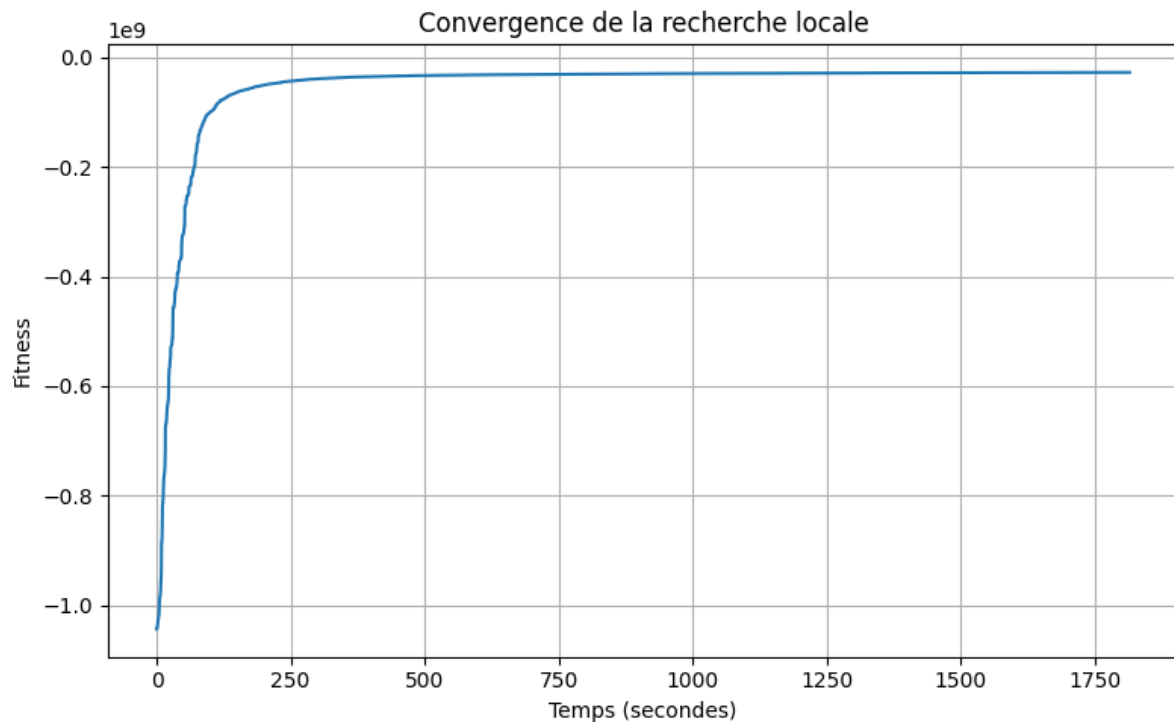
Avec 118 formes, nous eûmes, toujours au bout de 30 minutes,



->



Pour une convergence (nous mîmes au point un outil de visualisation de convergence et de sauvegarde du graphique)



Il faut préciser que ces tests-là se faisaient encore, à ce moment-là, sur l'un de nos ordinateurs qui se trouvait être le moins performant. Avec un ordinateur meilleur et un peu plus proche de la moyenne standard actuelle en termes de performances, le résultat aurait probablement été meilleur.

Cela dit, cette contrainte de performance nous donna une idée. Plutôt que de recalculer à chaque fois la fitness avec toute la matrice des deux images, pourquoi ne pas trouver le moyen d'uniquement recalculer sur la zone qui a subi la perturbation ? Un tel procédé nous permettrait de diviser par 10, voire plus, le temps de calcul.

Mais avant de nous pencher sur cette astuce miracle, nous entreprîmes de mettre au point non seulement le parser qui était attendu en rendu, mais aussi un système de sauvegarde de génotype et de reprise de recherche locale sur sauvegarde.

*Mise en place du parser

```
warnings.filterwarnings("ignore")
parser = argparse.ArgumentParser(description='IArt-Geom Pipeline')
parser.add_argument('--shape',
                    choices=['ellipse', 'triangle', 'rectangle'], default='ellipse', help='Which shape do you want ?')
parser.add_argument('--n', default=118, type=int, help='How many shapes ?')
parser.add_argument('--time', default=30, type=int, help='In how much time max ? (in seconds)')
parser.add_argument('--input', default='mona.jpg', type=str, help='Input image name ?')
parser.add_argument('--output', default='mutated_phenotype', type=str, help='Output image name ? (without the extension)')
parser.add_argument('--from_gen', default=None, type=str, help='The name of the genotype file (If you have one to start from).')
```

Comme le montre la figure, nous avons non seulement intégré les paramètres attendus, mais également un paramètre de reprise sur sauvegarde. Nous reviendrons sur ce dernier dans la sous-section suivante.

Plutôt que de simplement mettre en place un parser bête et méchant qui n'a rien d'autre pour se venter que des valeurs par défaut spécifiées, nous l'avons doté de gardes fous. Pour contrôler les valeurs numériques

```
25 if not (118 <= args.n <= 20000):
26     parser.error("--n must be between 118 and 20000")
27
28 if not (30 <= args.time <= 3000):
29     parser.error("--time must be between 30 and 3000")
```

comme le nombre de formes et le temps, afin d'éviter les valeurs aberrantes ; mais aussi afin de contrôler le choix des formes. De façon générale, nous avons fait en sorte que le parser assiste l'utilisateur afin de l'aider à entrer la ligne de commande attendue par lui, mais aussi par le système. C'est un parser assistant.

Comme promis, parlons à présent du système de reprise sur sauvegarde.

*Reprise sur sauvegarde

Comme l'indique la présence du paramètre dédié, il existe une fonction de reprise sur sauvegarde. Si le fichier de reprise est fourni, le génome soumis est chargé, et c'est à partir de lui que la recherche locale commence. Cela peut être extrêmement pratique, surtout si l'on vise une longue recherche sans devoir se contraindre de nécessité de contiguïté du temps de calcul.

D'un autre côté, la fonction principale du programme a évolué pour se parer d'une capacité de sauvegarde. A chaque époque, le génome (forcément alors le meilleur trouvé jusque-là, comme le veut notre logique), est sauvegardé dans le fichier de sauvegarde dédié (indiqué en paramètre). Le même fichier est écrasé à chaque fois afin de ne garder que la version récente. Ce fichier pourra alors allègrement être utilisé ultérieurement pour toute reprise. Cette sauvegarde est permise grâce à la sérialisation de l'objet python représentant notre génome, sérialisation permise par la bibliothèque 'pickle'.

Nous disposons alors d'un programme efficace, mais aussi résilient.

Et à cela, s'ajoute en fait, désormais, une vitesse de calcul décuplée.

****Optimisation du calcul de fitness (drastique)**

Comme évoqué quelques lignes plus tôt, nous avons pensé à une stratégie qui nous évite de devoir recalculer la fitness avec l'entière des matrices d'images à chaque fois. Nous ne recalculerions que dans la zone qui s'est vue subir la permutation. Et c'est exactement cette idée que nous matérialisâmes. Voici comment nous nous y prîmes :

La première étape consista à ajouter à chaque forme (classe de forme) une méthode '**get_bounds**'.

Elle a pour rôle de fournir les coordonnées de la zone rectangulaire le plus petit possible contenant la forme. On fait appel pour nous aider en cela à '**boundingRect()**' de openCv notamment (sachant que dans le cas des ellipses, il a fallu un peu ruser).

Il a par la suite s'agit de faire évoluer notre stratégie de calcul de fitness en ajoutant une fonction '**fitness_local_change**'. Elle calcule la variation de fitness uniquement sur la zone modifiée (bbox), et retourne la différence de score (Delta).

Une autre fonction fut faite : **get_union_bounds**

Elle s'occupe de la tâche de calculer le rectangle englobant de deux de deux rectangles.

Enfin, la dernière étape, la plus délicate, fut de mettre à jour la logique générale du code (surtout les parties calculant et utilisant la fitness) de notre fonction principale pour refléter nos changements.

Pour comprendre à quel point cette optimisation est efficace, il suffit de penser à un cas de figure où une perturbation aurait effectué une légère rotation sur une ellipse de 10px de rayon, dans un cadre total de $250*250 = 62500$ px.

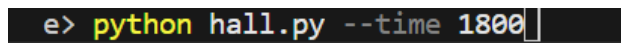
L'ancienne méthode aurait recomparé tous l'entière des deux matrices du haut de leurs dizaines de milliers de pixels, tous les calculs inclus.

Avec la nouvelle, et avec une boundingbox de seulement $20*20 = 40$ px, seule la zone de 40px sert de contexte de calcul.

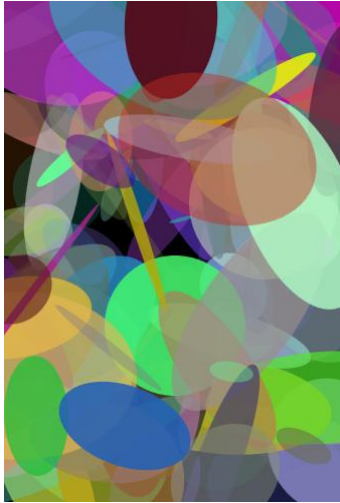
Un gain phénoménal qui permet de faire près de 150 fois moins de calculs. La différence de performance en est drastique.

En tout cas, en théorie. Qu'en est-il en réalité de la pratique ? L'ajout de plusieurs fonctions supplémentaires pourrait-il être contre-productif dans les faits ?

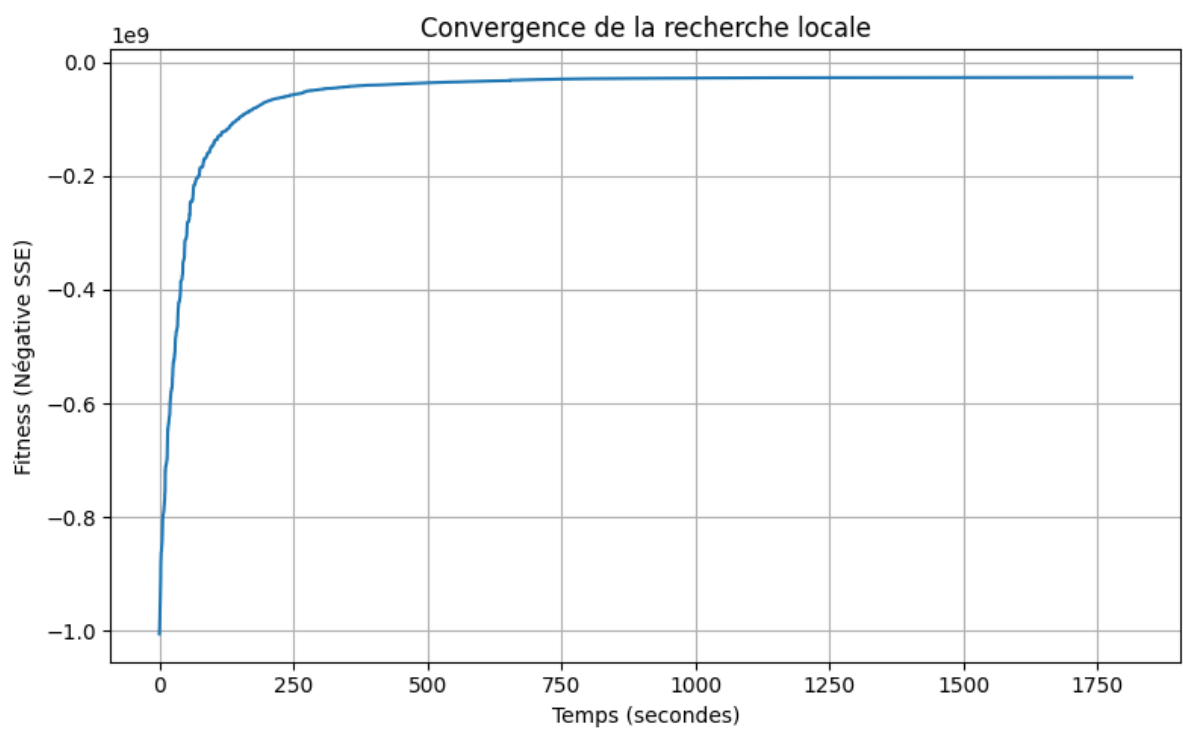
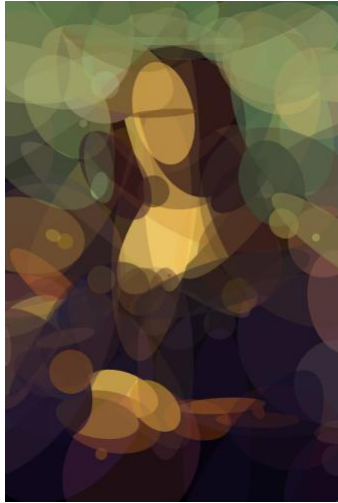
Nous devons en avoir le cœur net. Il fallait tester.

 (118 ellipses, 30 minutes, sans fichier de reprise, exactement les mêmes paramètres que ceux utilisés pour le précédent test).

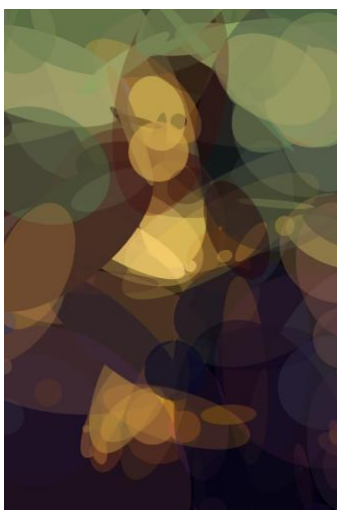
Le résultat se veut sans appel. D'autant plus que là où cette fois nous finissons avec -26932586 de fitness, donc une erreur de 26932586, le cas précédent nous en offrait une de 36932586.



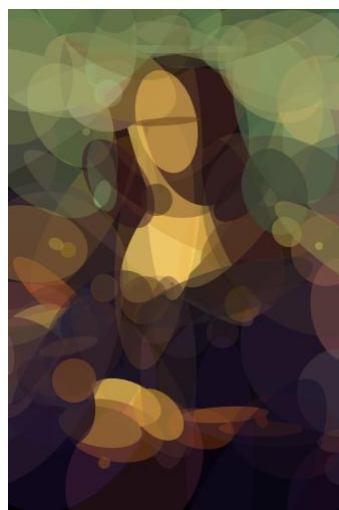
->



:



vs



(Originale : Avant vs Après)

Il est important de dire que nous étions animés d'une grande volonté de tester, dans le cadre de ce projet, plusieurs autres méthodes de recherche locale comme le recuit simulé par exemple que nous trouvons fascinant. Mais il fallait clairement se rendre à l'évidence, nous manquions de temps. Les échéances de date du rendu du projet, en plus de toutes les évaluations que nous devions préparer en cette période ne nous permirent pas d'aller plus loin. Ce fut frustrant.

Néanmoins, ce n'est pas pour autant que notre exploration s'arrêta-là.

Dans les lignes qui suivent, nous relateront, de façon cela dit plus succincte que précédemment, notre exploration d'autres contrées de solution (dont l'une consiste, par exemple, à jouer sur la résolution de l'image source et à améliorer celle-ci au fur et à mesure de façon stratégique).

V- L'EXPLORATION

Arrivés à ce stade du projet, nous avons fait un constat clair : malgré une base algorithmique désormais solide, l'approche suivie jusqu'ici atteignait rapidement ses limites. Les améliorations devenaient marginales, coûteuses en temps de calcul, et souvent invisibles à l'œil humain. C'est à partir de ce point que nous avons pris une décision forte : repartir de zéro.

Il ne s'agissait pas d'une simple continuation ou d'un raffinement incrémental de l'approche précédente (Partie A), mais bien d'une remise à plat complète de la modélisation, des hypothèses et des choix algorithmiques. Cette transition marque ainsi un changement net de posture : là où la Partie A explorait une première voie possible, la Partie B assume explicitement une démarche expérimentale indépendante, guidée par l'analyse critique des échecs précédents plutôt que par leur prolongement. Cette nouvelle branche de travail, que nous appellerons par la suite *Partie B*, a été menée indépendamment, avec un regard critique sur les échecs et blocages rencontrés jusque-là.

***Retour aux fondations : une nouvelle modélisation**

Nous avons commencé par redéfinir formellement le problème, en nous forçant à expliciter chaque composant, sans réutiliser implicitement les décisions antérieures.

- L'image cible est définie comme une matrice de pixels : $I \in [0,255]^{H \times W \times 3}$
- Une solution est une liste ordonnée de formes géométriques : $S=[s_1, s_2, \dots, s_N]$
- Chaque forme étant décrite par un ensemble minimal de paramètres : (type, position, paramètres de taille, rotation, couleur, alpha).
- Le phénotype associé à une solution est obtenu en dessinant les formes dans l'ordre sur un canevas initial, produisant une image P_S .
- La fonction objectif consiste à minimiser l'erreur quadratique par pixel :
- $f(S) = \sum_{x,y} \| I(x,y) - P_S(x,y) \|^2$

Si cette formulation semble classique, elle a néanmoins servi de point d'ancrage pour remettre en question les choix d'optimisation effectués jusque-là.

*Choix algorithmique initial : Iterated Local Search

Dans cette nouvelle branche de travail, nous avons fait le choix assumé de repartir sur un cadre algorithmique volontairement simple mais finement maîtrisé : l'Iterated Local Search (ILS). Ce choix ne relevait pas d'un manque d'ambition algorithmique, mais d'une volonté de garder un contrôle total sur les mécanismes internes de l'optimisation, afin de mieux comprendre les causes réelles des échecs observés précédemment.

Plusieurs arguments ont motivé cette décision. Tout d'abord, l'ILS converge rapidement vers des solutions cohérentes sur le plan visuel, ce qui permet d'obtenir très tôt un retour qualitatif sur le comportement de l'algorithme. Ensuite, sa structure séquentielle rend son fonctionnement particulièrement lisible : chaque amélioration est directement attribuable à une mutation identifiable. Enfin, son excellent comportement anytime est parfaitement adapté au cadre du TP, dans lequel le temps de calcul constitue une contrainte centrale.

Concrètement, l'algorithme repose sur une initialisation gloutonne, produisant une première solution acceptable, suivie d'une boucle d'amélioration locale. Les mutations sont appliquées forme par forme (position, taille, rotation, couleur, opacité), avec une stratégie d'acceptation à la première amélioration. Des perturbations plus larges sont ponctuellement introduites afin de tenter de sortir des minima locaux.

Malgré une implémentation correcte et plusieurs raffinements (mutations adaptatives, meilleure initialisation, optimisation du calcul de fitness), les résultats obtenus sont rapidement apparus limités. Les améliorations mesurées numériquement ne se traduisaient que très peu visuellement, révélant des limites plus profondes que de simples choix de paramètres.



Image source : Mona Lisa
cette étape, 100 formes en 60 secondes

->

Image approximée par l'algorithme à

**Premiers diagnostics : le problème du paysage de fitness

Les versions successives (v1 à v3) ont mis en évidence un phénomène récurrent : alors même que la fitness diminuait de manière mesurable, la qualité visuelle de l'image stagnait. Ce décalage entre optimisation numérique et perception humaine a constitué un signal d'alerte fort.

L'analyse a permis d'identifier plusieurs causes structurelles. La fonction de fitness globale, en agrégeant l'erreur sur l'ensemble des pixels, masquait systématiquement les améliorations locales pertinentes. Une mutation pouvait améliorer significativement une petite zone, mais être rejetée car elle dégradait marginalement de nombreuses autres régions.

Par ailleurs, les formes utilisées couvraient souvent une portion trop importante de l'image. Toute modification géométrique affectait alors un grand nombre de pixels, rendant le coût d'une mutation trop élevé au regard du gain local espéré. L'algorithme se retrouvait ainsi piégé dans des plateaux locaux, incapable de franchir le cap nécessaire au raffinement.

En pratique, l'ILS optimisait correctement une approximation grossière de l'image, mais se révélait structurellement inadapté pour faire émerger des détails fins. Même en augmentant fortement le temps de calcul, l'algorithme demeurait bloqué dans cet état intermédiaire.

*Tentatives d'amélioration : multi-résolution et fitness locale

Afin de dépasser ces blocages, plusieurs stratégies plus élaborées ont été explorées. La première repose sur une approche multi-résolution : l'image est d'abord optimisée à basse résolution afin de capturer les grandes structures globales, puis progressivement raffinée à des résolutions plus élevées. Cette méthode vise à lisser le paysage de fitness et à éviter certains minima locaux précoces.

En parallèle, nous avons introduit une fitness locale, calculée uniquement sur la zone effectivement impactée par une mutation (bounding box ou patch local). Cette modification avait un double objectif : rendre visibles pour l'algorithme des améliorations locales pertinentes, et réduire drastiquement le coût de calcul afin d'augmenter le nombre d'itérations possibles.

Ces ajustements ont effectivement amélioré les performances computationnelles, mais n'ont pas résolu le problème de fond. Même avec une fitness plus locale et une optimisation hiérarchique en résolution, nous cherchions toujours à améliorer l'image entière, une forme à la fois. Le paradigme global restait inchangé, et ses limites devenaient de plus en plus évidentes.

**Changement de paradigme : optimisation par résidu

Face à ces constats, un changement de paradigme s'est imposé. Plutôt que d'optimiser chaque forme par rapport à l'image cible entière, nous avons choisi d'optimiser chaque nouvelle forme par rapport à ce qu'il reste à expliquer : le résidu.

À chaque étape (k), nous définissons le résidu comme la différence entre l'image cible et le canevas courant : $R_k = I - P_{k-1}$

La nouvelle forme est alors optimisée pour approximer ce résidu : $s_k \approx \operatorname{argmin} \|s - R_k\|$

Dans la pratique, l'optimisation géométrique est découplée de la couleur : la couleur et l'opacité de la forme sont directement échantillonnées depuis l'image cible sous la forme candidate. Ce choix réduit considérablement l'espace de recherche et stabilise fortement l'optimisation.

Cette approche permet d'éviter les dérives chromatiques, garantit une amélioration monotone de l'image, et correspond bien mieux à une construction progressive de l'approximation.





Ci-dessus la progression de notre algorithme à ce moment, avec à gauche la source, au centre l'approximation obtenue avec la version précédente (100 formes, 60 secondes), et à droite la version mettant en place les améliorations pensées (150 formes, 60 secondes).

****Détour : l'approche constructive gloutonne**

À ce stade du projet, nous avons entrepris un détour conceptuel volontaire, sans lien direct avec l'architecture en cours de développement. L'objectif n'était pas d'améliorer l'algorithme existant, mais de tester un paradigme radicalement différent afin de mieux comprendre ce qui rend certaines approches efficaces en pratique.

Nous avons ainsi implémenté une approche constructive gloutonne, inspirée de logiciels spécialisés comme *Geometrize*. Le principe est simple : au lieu de placer un ensemble de formes puis de tenter de les optimiser globalement, l'algorithme cherche, à chaque étape, la meilleure forme unique qui réduit le plus l'erreur actuelle.

Une fois cette forme trouvée, elle est fixée définitivement sur le canevas, et l'on recommence sur le résidu restant. Les premières formes capturent naturellement les grandes zones de couleur et les structures globales, tandis que les formes suivantes, de plus en plus petites, viennent corriger les détails fins.

Cette approche présente plusieurs propriétés intéressantes : elle garantit une amélioration monotone de l'image, rend le comportement de l'algorithme très lisible, et permet d'obtenir rapidement une approximation visuellement convaincante. En revanche, atteindre une très grande précision nécessite un nombre élevé de formes, ce qui la rend moins adaptée à ce TP.

Ce détour s'est néanmoins révélé extrêmement instructif et a profondément influencé les choix effectués par la suite.





Ci-dessus, l'approximation par la version constructive avec 200 formes (image droite) de la Mona Lisa (image gauche)

*Retour à l'optimisation : sculpture plutôt que peinture

Dans un dernier temps, nous avons tenté de réintroduire une phase d'optimisation globale, mais avec une philosophie différente :

- toutes les formes sont générées dès le départ,
- aucune nouvelle forme n'est ajoutée par la suite,
- l'algorithme ne fait que déplacer, redimensionner, recolorier ou réordonner les formes.

Cette approche dite de *sculpture* a été combinée avec :

- du recuit simulé,
- des mutations adaptatives,
- une fitness locale basée sur la bounding box,
- des mutations de superposition (z-index),
- des mécanismes de téléportation ponctuelle.

**Résultats et limites

Les dernières versions de l'algorithme (v8 à v10) produisent des approximations globales de bonne qualité en un temps limité (60 secondes). Les structures principales de l'image sont correctement positionnées, les grandes masses de couleur sont cohérentes, et le sujet est immédiatement reconnaissable à l'œil humain.

Toutefois, les détails fins restent coûteux à obtenir. Leur apparition nécessite un nombre très élevé d'itérations, et donc plusieurs minutes de calcul. Cette observation met en évidence une limite intrinsèque du compromis temps / précision imposé par le cadre du TP : améliorer rapidement la structure globale est relativement aisé, mais raffiner localement l'image jusqu'à un niveau élevé de fidélité devient rapidement prohibitif.

Ces résultats restent néanmoins satisfaisants au regard des contraintes imposées et illustrent clairement l'impact déterminant des choix de modélisation et de paradigme d'optimisation sur la qualité finale obtenue.





Ci-dessus, la progression de notre algorithme au travers du temps, avec à gauche la source, au centre l'approximation obtenue avec une des premières versions (100 formes, 60 secondes), et à droite la plus récente version mettant au point notre nouvelle philosophie (150 formes, 60 secondes).

**Illustrations finales

Les figures intégrées tout au long de cette partie ont été choisies afin d'illustrer précisément les ruptures conceptuelles majeures du projet. Pour compléter ces exemples, nous proposons un parcours de notre dernière version sur l'approximation de la Mona Lisa, de 1 minute à 15 heures d'approximation, illustrant le comportement asymptotique de l'algorithme lorsque la contrainte de temps est relâchée.



Source

Approximation 1 minute

Approximation 10 minutes

Approximation 1 heure

Cette exploration, bien que jalonnée d'échecs et de détours, a profondément enrichi notre compréhension du problème. Elle met en évidence l'importance du choix de la représentation, de la fonction objectif, et surtout du paradigme d'optimisation, souvent plus déterminant que l'algorithme lui-même.

CONCLUSION

Avant nos mots de fins, il pourrait être le lieu congru de partager une anecdote – que l'on pourrait hésiter à qualifier soit de dramatiquement amusante ou de comiquement dramatique – relative à notre projet : celle des 15 heures de recherche perdues. En effet, comme il pourrait se laisser deviner, nous eûmes à lancer, par le biais de notre solution B, une recherche locale d'une durée paramétrée à 15 heures. Le résultat d'un tel temps de recherche nous rendait absolument curieux, autant curieux qu'impatients. Il ne pourrait être qu'impressionnant. Sauf qu'au bout de 5 heures, l'ordinateur crasha, et tout fut perdu. Comme, contrairement au cas de la solution A, nous n'avions pas pris le temps de mettre en place de mécanisme de reprise sur sauvegarde pour notre solution B, la perte fut définitive. Ce fut un constat à la fois amer et plein de leçons.

Pour conclure, que dire d'autre que ce fut inéluctablement une aventure exceptionnelle, autant enrichissante que passionnante. L'informatique mise au service de l'art nous a dévoilé des possibilités faramineuses, qui nous ont d'ailleurs donné maintes idées de projets personnels. Une chose est sûre, même une fois notre rendu effectué, nos recherches locales ou génétiques ; guidées par la boussole d'optimisation de fitness, par les vents de recherche du rendu esthétique le plus satisfaisant, sur les chemins, voies et routes de l'art pensé en 0 et en 1 ; n'auront pas pris fin avec ce point final de rapport.